

Manuel LuaPilot

Table des matières

LuaPilot — Manuel utilisateur	4
Pour démarrer	4
Modules	4
Cookbook	5
Génération du PDF	5
Conventions utilisées dans la doc	5
Pour démarrer	5
Téléchargement	5
Compilation depuis les sources	6
Trois manières de lancer un script Lua	6
1. Mode dossier	6
2. Mode embarqué (binaire autonome)	6
3. Embarqué via PATH (dossier + auto-détection)	6
Premier script	6
Pour aller plus loin	7
Modèle de sécurité	7
Ce contre quoi le durcissement <i>protège</i>	7
Ce contre quoi il <i>ne protège pas</i>	8
Pattern recommandé : moindre privilège	8
argparse — parseur d’arguments CLI (module Lua)	8
Pourquoi	8
API	8
Exemple rapide	9
Contrat d’erreur	9
Décisions de design	9
Hors v1	9
luapilot fs — système de fichiers	10
Pourquoi	10
API — Existence, type, attributs	10
API — Création, suppression, déplacement	10
API — Listing et recherche	10
API — Copie	11
API — Hashes et checksums	11
Exemples rapides	11
Contrat d’erreur	12
Décisions de design	12
Hors v1	12
luapilot.http — client HTTP	12
Pourquoi	12
API	12

Table <code>opts</code>	13
Table <code>response</code>	13
Exemples rapides	13
Contrat d'erreur	14
TLS / trust store	14
Décisions de design	14
Hors v1	14
luapilot.inotify — surveillance d'événements fichier	14
Pourquoi	15
API	15
Argument <code>events</code>	15
Événement renvoyé par <code>read</code>	15
Exemple rapide	16
Contrat d'erreur	16
Décisions de design	16
Hors v1	16
luapilot.json — encode/décode JSON	17
Pourquoi	17
API	17
Exemple rapide	17
Contrat d'erreur	17
Décisions de design	18
Hors v1	18
logging — logger avec niveaux (module Lua)	18
Pourquoi	18
API	18
Exemple rapide	18
Contrat d'erreur	19
Décisions de design	19
Hors v1	19
luapilot.signal — signaux POSIX	19
Pourquoi	19
API	19
Exemple rapide	20
Comment les callbacks s'exécutent	20
Contrat d'erreur	21
Décisions de design	21
Hors v1	21
luapilot.socket — sockets TCP client et serveur	21
Pourquoi	21
API	21
Constructeurs	21
Méthodes (socket client)	22
Méthodes (socket serveur)	22
Exemples rapides	22
Client TCP echo	22
Serveur line-protocol simple avec arrêt propre	22
Contrat d'erreur	23
Décisions de design	23
Hors v1	23
luapilot.sqlite — base de données SQL embarquée	23

Pourquoi	23
API	24
Ouverture et fermeture	24
Exec direct (pas de paramètres / instruction one-shot)	24
Instructions préparées (réutilisation, performance)	24
Transactions	24
Utilitaires	24
Mapping de types	25
Exemples rapides	25
Contrat d'erreur	26
Décisions de design	26
Hors v1	26
luapilot strings — manipulation de chaînes	26
Pourquoi	26
API	26
Exemple rapide	27
Contrat d'erreur	27
Décisions de design	27
Hors v1	27
luapilot sys — utilitaires système	27
Pourquoi	27
API	27
Exemple rapide	27
Contrat d'erreur	28
Décisions de design	28
Hors v1	28
luapilot tables — helpers de manipulation de tables	28
Pourquoi	28
API	29
Exemple rapide	29
Contrat d'erreur	29
Décisions de design	29
Hors v1	30
luapilot.time — horloges monotone et temps réel	30
Pourquoi	30
API	30
Exemple rapide	30
Contrat d'erreur	31
Décisions de design	31
Hors v1	31
luapilot.socket TLS — connect_tls et starttls	31
Pourquoi	31
API	31
Exemples rapides	32
Connexion à IRC over TLS	32
Upgrade STARTTLS (soumission SMTP)	32
Serveur auto-signé (dev / CA privée)	32
Skip de la vérification (TESTS UNIQUEMENT)	32
Contrat d'erreur	32
Trust store	33
Décisions de design	33
Hors v1	33

luapilot.toml — parseur TOML	33
Pourquoi	33
API	33
Exemple rapide	34
Contrat d’erreur	34
Décisions de design	34
Hors v1	34
luapilot.user	34
Pourquoi	35
API	35
Table renvoyée en cas de succès	35
Exemple rapide	35
Contrat d’erreur	35
Décisions de design	36
Hors v1	36
luapilot.workers — jobs parallèles	36
Pourquoi	36
API	36
Argument <code>code</code>	36
Argument <code>args</code>	37
Exemples rapides	37
Fetches HTTP parallèles	37
Travail CPU-bound avec cancellation	37
Pattern pool de workers	38
Contrat d’erreur	38
Partage de données	38
Décisions de design	39
Hors v1	39
Cookbook	39
Surveiller un dossier et envoyer les nouveaux fichiers par email	39
S’assurer qu’un user système existe, puis basculer dessus	40
Récupérer beaucoup d’URLs en parallèle	40
Arrêt propre d’un script long-running	40
Lire une config TOML, valider, persister en SQLite	41
Pretty-print d’une valeur Lua	41
Hello there	42

[English](#) | **Français**

LuaPilot — Manuel utilisateur

LuaPilot est un binaire Lua 5.5 standalone pour le scripting et l’automatisation sous Linux. Ceci est le manuel de référence, organisé en un fichier par module pour que chaque section reste ciblée et facile à maintenir.

Pour démarrer

- [getting-started.md](#) — installation, premier script, les trois modes de lancement (dossier / embarqué / via PATH).
- [security.md](#) — ce contre quoi le durcissement protège et ne protège pas, patterns de moindre privilège.

Modules

Chaque module vit dans son propre fichier sous `modules/` :

Module	Rôle
argparse	Parseur d'arguments de ligne de commande.
fs	Système de fichiers : listing, copie, hash, attrs.
http	Client HTTP (basé sur <code>cpp-httpplib</code>).
inotify	Surveillance d'événements fichier (<code>inotify(7)</code>).
json	Encode/décode JSON (<code>nlohmann/json</code>).
logging	Logger avec niveaux.
signal	Signaux POSIX : <code>SIGTERM</code> , <code>SIGINT</code> , ...
socket	TCP client/serveur avec timeouts.
sqlite	Base de données SQL embarquée.
strings	Manipulation de chaînes (<code>split</code> , etc.).
sys	<code>which</code> , <code>hostname</code> , <code>env</code> , <code>pid</code> , ...
tables	Manipulation de tables (<code>mergeTables</code> , <code>deepCopyTable</code>).
time	Horloges monotone et temps réel.
tls	Sockets TLS : <code>connect_tls</code> , <code>starttls</code> .
toml	Parseur TOML (<code>tomlplusplus</code>).
user	Lookups d'utilisateurs système via NSS.
workers	Jobs parallèles (vrais threads OS).

Le module [user](#) est la référence canonique du pattern de documentation : *Pourquoi*, *API*, *Exemple rapide*, *Contrat d'erreur*, *Décisions de design*, *Hors v1*. Les autres modules varient dans le respect strict de ce template ; l'objectif au fil du temps est de les aligner.

Cookbook

[cookbook.md](#) — recettes concrètes : watch de dossier + email, fetches HTTP parallèles, arrêt propre, TOML+SQLite, etc.

Génération du PDF

Le manuel peut être exporté en un seul PDF :

```
cd docs
make manual-fr.pdf # utilise pandoc + LaTeX
```

Voir [docs/manual_order_fr.txt](#) pour l'ordre des fichiers ; c'est le seul endroit à éditer quand on ajoute ou réordonne des chapitres.

Conventions utilisées dans la doc

- **Tables d'API** : chaque module commence par une petite table qui liste les fonctions, leurs signatures, et ce qu'elles renvoient.
- **Contrat d'erreur** : une section dédiée explique quelles erreurs sont levées (`luaL_error`) et lesquelles sont renvoyées en (`nil`, `err`). Cohérent entre tous les modules.
- **Décisions de design** : quand un choix n'est pas évident ("pourquoi pas récursif ?", "pourquoi obligatoire ?"), il y a un bref rationnel.
- **Hors v1** : une courte liste de ce qui pourrait être ajouté de manière additive plus tard sans casser le SemVer.

[English](#) | **Français**

Pour démarrer

Téléchargement

Des binaires précompilés pour x86_64, aarch64 (RPi4) et armv6l (RPi0) sont disponibles sur la [page des releases](#).

Compilation depuis les sources

LuaPilot embarque toutes ses dépendances (Lua, OpenSSL, SQLite, miniz, nlohmann/json, cpp-httplib, tomlplusplus), donc les seuls prérequis sur ton système sont :

- un compilateur C++23 (GCC ou Clang récent)
- CMake (3.20)
- wget et unzip

```
git clone https://github.com/Chipsterjulien/luapilot_standalone.git
cd luapilot_standalone
./build_local.sh           # télécharge les deps et compile
                           # (~5 min au premier lancement, plus vite après)
./run_tests.sh             # harness offline – doit afficher 827 PASS / 0 FAIL
```

Le script de build télécharge chaque dépendance depuis sa source upstream, vérifie son SHA256, puis compile statiquement. Si la source upstream est temporairement indisponible (lua.org a notamment connu des pannes), le script bascule sur la Wayback Machine d'Internet Archive — le check SHA256 reste appliqué.

Le binaire produit est dans `test/luapilot`.

Trois manières de lancer un script Lua

LuaPilot supporte trois modes de lancement :

1. Mode dossier

Pointe le binaire vers un dossier contenant `main.lua` :

```
echo 'print("hello from LuaPilot")' > main.lua
./test/luapilot .
```

`require("X")` dans `main.lua` cherche `X.lua` dans le même dossier.

2. Mode embarqué (binaire autonome)

Empaquète un script et ses modules dans un binaire autonome :

```
./test/luapilot --create-exe . myapp
./myapp           # lance main.lua depuis le ZIP embarqué
```

En interne, LuaPilot append un ZIP à son propre binaire et lit `main.lua` (plus tous les modules `require`) depuis ce ZIP au runtime. Le `myapp` résultant est totalement autonome — pas besoin d'avoir LuaPilot installé sur la machine cible, juste glibc.

3. Embarqué via PATH (dossier + auto-détection)

Si LuaPilot est sur `$PATH` et qu'on fait `chmod +x main.lua` après avoir ajouté une ligne shebang :

```
#!/usr/bin/env luapilot
print("hello")

chmod +x main.lua
./main.lua
```

LuaPilot utilise le dossier du script comme dossier de travail, donc `require("helpers")` trouvera `helpers.lua` à côté de `main.lua`.

Premier script

Une fois le binaire compilé, essaie ceci :

```
-- main.lua
print("LuaPilot dit bonjour")
print("PID :", luapilot.pid())
print("Hôte :", luapilot.hostname())

local r, err = luapilot.http.get("https://example.com/")
if r then
    print("Status :", r.status)
else
    print("Échec HTTP :", err)
end
```

Lance-le avec `./test/luapilot .` (ou un autre mode au choix).

Pour aller plus loin

- Chaque module a sa propre page sous `modules/`.
- Le module `user` est un petit exemple complet du pattern de documentation utilisé partout — lis-le comme la référence canonique.
- Vois `security.md` avant d'exposer des scripts LuaPilot à quoi que ce soit qui pourrait recevoir de l'input non fiable.

[English](#) | **Français**

Modèle de sécurité

LuaPilot exécute des scripts Lua de confiance. Ce n'est pas une sandbox.

Les scripts s'exécutent avec la bibliothèque standard Lua complète (`luaL_openlibs`), et LuaPilot expose en plus des primitives système comme `exec`, `remove`, `rmdirAll` et `chdir`. Un script a donc les mêmes privilèges que le processus qui le lance : il peut lire, modifier ou supprimer des fichiers, et lancer des commandes arbitraires. C'est volontaire — comme `make`, un script shell, ou n'importe quel outil de build, LuaPilot est conçu pour exécuter du code que tu contrôles.

Ne lance que des `main.lua` et des modules `require` auxquels tu fais confiance. **N'utilise pas** LuaPilot pour exécuter du Lua venant de sources non fiables ; il n'offre aucune isolation contre du code hostile, et n'est pas pensé pour. Si tu dois exécuter des scripts non fiables, utilise plutôt une sandbox Lua dédiée à cet usage.

Ce contre quoi le durcissement *protège*

Le travail de durcissement dans LuaPilot protège les usages *légitimes* contre les accidents et les attaques de supply chain :

- **Gestion des chemins / symlinks** utilise `lexically_relative()` et des guards `is_within()` composant par composant (jamais des checks de préfixe de string), donc un `cp src dst/` ne peut pas s'évader dans un arbre voisin via un composant symlinké.
- **Cleanup des groupes de processus** dans `luapilot.exec` garantit que les enfants forkés ne survivent pas au parent et ne laissent pas de processus zombies quand le script est interrompu.
- **Limites de sortie** sur `luapilot.exec` bornent la quantité de stdout et stderr capturés en mémoire (par défaut 16 MiB chacun, configurable), pour qu'un sous-processus emballé ne puisse pas OOM le processus LuaPilot.
- **Checksums des dépendances** : chaque dépendance embarquée est SHA256-pinnée dans `build_local.sh`. Un hash vide refuse la build. Un mismatch supprime le fichier téléchargé et sort en erreur. Ça protège contre un upstream compromis ou une archive Wayback Machine altérée (source de fallback).
- **Vérification TLS** est activée par défaut (`verify=true`). La désactiver demande un `verify=false` explicite par appel — pas de fallback silencieux. Des CA stores personnalisés peuvent être passés via `ca_cert` ou `ca_path`, ou via les variables d'environnement `SSL_CERT_FILE` / `SSL_CERT_DIR`.

Ce contre quoi il *ne protège pas*

- Un script malveillant. LuaPilot n'a pas de sandbox. Si tu fais `exec` sur de l'input utilisateur, tu as une injection shell. Si tu fais `loadstring` sur de l'input utilisateur, tu as une exécution de code arbitraire.
- Un attaquant déterminé qui a déjà obtenu une exécution de code sur la machine qui fait tourner LuaPilot. Le durcissement rend les erreurs accidentelles visibles, pas les attaques adversariales impossibles.
- Les problèmes OS-level (exploits kernel, évasions de conteneur, escalade de privilèges). LuaPilot est un binaire userland ordinaire.

Pattern recommandé : moindre privilège

Quand tu fais tourner LuaPilot en service, traite-le comme n'importe quel processus non privilégié :

```
# En root, crée un user système dédié, sans shell.
```

```
useradd --system \  
  --home-dir /var/lib/myapp \  
  --create-home \  
  --shell /usr/sbin/nologin \  
  --comment "myapp LuaPilot service" \  
  myapp
```

```
# Utilise luapilot.user.exists("myapp") dans ton script d'install  
# pour vérifier que le user est provisionné avant de lancer le  
# daemon.
```

Combine ça avec des options de unit `systemd` comme `User=myapp`, `PrivateTmp=yes`, `ProtectSystem=strict`, `NoNewPrivileges=yes`, et `CapabilityBoundingSet=` (vide sauf si tu as vraiment besoin d'une capability). Le kernel fait bien plus que LuaPilot ne peut le faire pour contenir un script qui se comporte mal ; laisse-le faire.

[English](#) | [Français](#)

argparse — parseur d'arguments CLI (module Lua)

Un parseur d'arguments déclaratif pour les scripts en ligne de commande. Embarqué en module Lua pur, chargé via `require("argparse")`. Inspiré de l'`argparse` de Python, adapté aux idiomes Lua.

Pourquoi

Un script LuaPilot qui prend des arguments ne devrait pas avoir à réinventer le parsing (positionnel vs flag, défauts, texte d'aide, coercion de type, flags répétés) à chaque fois. `argparse` rend les patterns CLI courants accessibles en une ligne.

API

```
local argparse = require("argparse")  
local p = argparse("nom-script", "Description en une ligne")
```

Méthode	Effet
<code>p:argument(name)</code>	déclare un argument positionnel
<code>p:option(name)</code>	déclare un flag (<code>-x / --xxx</code>) qui prend une valeur
<code>p:flag(name)</code>	déclare un flag booléen (sans valeur)
<code>p:parse()</code>	parse la table <code>arg</code> , renvoie la table parsée ou appelle <code>os.exit(1)</code> en erreur

Chaque déclaration renvoie un builder chaînable :

- `:description("texte")` — texte affiché dans `--help`.

- `:default(value)` — défaut si non fourni.
- `:convert(fn)` — fonction pour coercer la string brute (`tonumber`, etc.).
- `:count("*"|"+"|"?" )` — flags répétés; défaut une fois.
- `:choices({...})` — restreint à un ensemble fini.

Exemple rapide

```
local argparse = require("argparse")

local p = argparse("myapp", "Traite des fichiers.")
p:argument("input"):description("Fichier d'entrée")
p:option("-o --output"):description("Fichier de sortie"):default("out.txt")
p:option("-n --count"):description("Combien"):convert(tonumber):default(1)
p:flag("-v --verbose"):description("Sortie verbeuse")

local args = p:parse()
print(args.input, args.output, args.count, args.verbose)
```

À l'exécution :

```
$ ./myapp.lua data.csv --count 5 -v
data.csv  out.txt  5  true
```

```
$ ./myapp.lua --help
Usage: myapp [-o <output>] [-n <count>] [-v] <input>
```

Traite des fichiers.

Arguments:

input	Fichier d'entrée
-------	------------------

Options:

-o, --output	Fichier de sortie (default: out.txt)
-n, --count	Combien (default: 1)
-v, --verbose	Sortie verbeuse
-h, --help	Show this help message and exit

Contrat d'erreur

- **Construction du parser** (`argparse(...)`) et déclarations (`p:argument(...)` etc.) : lèvent via `error()` en cas de mauvais usage.
- **`p:parse()`** : sur une ligne de commande malformée, affiche l'erreur + l'usage sur `stderr` et appelle `os.exit(1)`. Cohérent avec les attentes utilisateurs pour les outils CLI (pas besoin de gérer les erreurs dans le script).

Décisions de design

- **Embarqué en Lua, pas en C++**. Le parsing d'arguments est de la logique string pure, pas de syscalls. Le Lua pur le garde monkey-patchable pour des besoins inhabituels (format d'aide personnalisé, etc.).
- **Exit dur en erreur de parse**. Les scripts CLI veulent typiquement mourir avec un message d'usage, pas attraper les erreurs. Si tu dois les attraper, wrappe l'appel dans `pcall`.
- **-h / --help est automatique**. Toujours présent, affiche l'usage et exit 0.

Hors v1

- Sous-commandes (style `git commit` / `git push`). Peut être ajouté si un cas d'usage réel apparaît.
- Groupes d'arguments (regroupement visuel dans l'aide). Cosmétique seulement.

L'implémentation de référence est embarquée depuis le projet upstream [argparse.lua](#). Voir sa doc pour les patterns avancés (groupes mutuellement exclusifs, actions personnalisées, etc.).

luapilot fs — système de fichiers

Un ensemble plat d'opérations système de fichiers exposées directement sur `luapilot` (pas de sous-namespace, précède la convention). Couvre les besoins quotidiens que `os.*` et `io.*` de Lua ne gèrent pas : listing récursif, copie avec attributs, hashing, attributs, liens.

Pourquoi

`io` et `os` de Lua te donnent `open`, `rename`, `remove`, et c'est à peu près tout. Tout ce qui va au-delà — lister un dossier, copie récursive, calculer un hash, interroger `mtime` — demande des programmes externes ou du FFI bas niveau. `luapilot` rassemble ça dans un set unique de fonctions qui se comportent de manière cohérente (chemins en string, erreurs en `(nil, "...")`).

Tous les chemins sont acceptés en string. L'expansion du tilde (`~/x`) n'est **pas** automatique — l'appelant doit résoudre `$HOME` lui-même si besoin. Les fonctions n'expansent jamais les globs ; pour le support glob, utilise `luapilot.find` avec un pattern explicite.

API — Existence, type, attributs

Fonction	Renvoie
<code>luapilot.fileExists(path)</code>	<code>boolean</code>
<code>luapilot.isFile(path)</code>	<code>boolean</code> (true seulement pour fichiers réguliers)
<code>luapilot.isdir(path)</code>	<code>boolean</code>
<code>luapilot.fileSize(path)</code>	<code>integer</code> octets <code>(nil, err)</code>
<code>luapilot.attributes(path)</code>	table avec <code>mtime</code> , <code>mode</code> , <code>size</code> , <code>uid</code> , <code>gid</code> , <code>inode</code> , etc. <code>(nil, err)</code>
<code>luapilot.symlinkattr(path)</code>	même shape que <code>attributes</code> , mais <code>lstat</code> (ne suit pas les symlinks) <code>(nil, err)</code>
<code>luapilot.mode(path)</code>	<code>integer</code> octal <code>(nil, err)</code> — raccourci pour <code>attributes(path).mode</code>

API — Création, suppression, déplacement

Fonction	Renvoie
<code>luapilot.touch(path)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — crée le fichier ou met à jour <code>mtime</code>
<code>luapilot.mkdir(path, opts?)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — <code>opts.parents = true</code> pour <code>mkdir -p</code>
<code>luapilot.rmdir(path)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — seulement dossiers vides
<code>luapilot.rmdirAll(path)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — récursif
<code>luapilot.remove(path)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — fichier ou symlink
<code>luapilot.rename(old, new)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>luapilot.chdir(path)</code>	<code>(true, nil)</code> <code>(nil, err)</code>
<code>luapilot.currentDir()</code>	<code>string</code> (absolu)
<code>luapilot.joinPath(a, b, ...)</code>	<code>string</code> — comme <code>path/a/b/c</code>
<code>luapilot.link(target, link, opts?)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — <code>opts.symbolic = true</code> pour symlink

API — Listing et recherche

Fonction	Renvoie
<code>luapilot.listdir(path)</code>	table (array des noms d'entrées, sans . / ..) (nil, err)
<code>luapilot.listFiles(path)</code>	table (seulement fichiers réguliers) (nil, err)
<code>luapilot.find(path, opts)</code>	fonction iterator, voir ci-dessous
<code>luapilot.fileIterator(path)</code>	fonction iterator sur les entrées du dossier

`find` accepte `opts` :

- `pattern = "*.lua"` — glob shell appliqué au nom de fichier seul.
- `recursive = true` — descend dans les sous-dossiers.
- `type = "f" / "d"` — restreint aux fichiers ou dossiers.
- `max_depth = N` — limite la profondeur de récursion.

API — Copie

Fonction	Renvoie
<code>luapilot.copy(src, dst, opts?)</code>	(true, nil) (nil, err)
<code>luapilot.copyTree(src, dst, opts?)</code>	(true, nil) (nil, err) — récursif
<code>luapilot.moveTree(src, dst)</code>	(true, nil) (nil, err)

Options de `copy` :

- `preserve = true` — garde mtime, mode, owner si possible.
- `overwrite = true` — remplace la destination si elle existe.

API — Hashes et checksums

Empreintes du contenu des fichiers. Résultat en **string hex minuscule** sauf mention contraire.

Fonction	Algorithme
<code>luapilot.md5(path)</code>	MD5
<code>luapilot.sha1(path)</code>	SHA-1
<code>luapilot.sha256(path)</code>	SHA-256
<code>luapilot.sha384(path)</code>	SHA-384
<code>luapilot.sha512(path)</code>	SHA-512
<code>luapilot.sha3_256(path)</code>	SHA3-256
<code>luapilot.sha3_384(path)</code>	SHA3-384
<code>luapilot.sha3_512(path)</code>	SHA3-512
<code>luapilot.blake2s(path)</code>	BLAKE2s-256
<code>luapilot.blake2b(path)</code>	BLAKE2b-512

Chaque renvoie `string` (hex) ou (nil, err). MD5 et SHA-1 sont exposés pour la compatibilité (Git, systèmes legacy) — ne pas les utiliser pour de nouveaux usages cryptographiques.

Exemples rapides

```
-- Prédicats
if luapilot.isdir("/etc") and luapilot.fileExists("/etc/hostname") then
    print("taille :", luapilot.fileSize("/etc/hostname"))
end

-- Listing récursif
```

```

for entry in luapilot.find("/var/log", { pattern = "*.log", type = "f", recursive = true }) do
    print(entry)
end

-- Copie avec attributs
luapilot.copyTree("./src", "/tmp/backup", { preserve = true, overwrite = true })

-- Hash d'une tarball de release
local sha, err = luapilot.sha256("/tmp/luapilot-1.7.0.tar.gz")
if sha then print("SHA256 :", sha) end

```

Contrat d'erreur

- Toutes les fonctions suivent la convention `(nil, err)` en cas d'échec runtime (fichier inexistant, permission refusée, etc.).
- Les mauvais **types** d'argument lèvent via `luaL_error` (passer un nombre où un chemin est attendu après coercion a une sémantique bizarre).
- Sécurité des chemins : `copy`, `copyTree`, `moveTree` utilisent `lexically_relative()` et des guards `is_within()` composant par composant pour empêcher des évasions par symlink hors de l'arborescence de destination. Voir [security](#).

Décisions de design

- **Namespace plat** : `fileExists` plutôt que `fs.exists`. Raison purement historique — ces fonctions précèdent la convention par module. Stable maintenant ; renommer casserait tous les scripts.
- **Les hashes opèrent sur des chemins de fichiers**, pas des octets bruts. Le cas courant est “hash ce fichier”, donc l'API prend un chemin. Pour les octets bruts, l'implémentation basée openssl est interne — pull request bienvenue si un cas d'usage émerge.
- **copy est mono-fichier, copyTree est récursif**. Lua n'a pas la surcharge de fonctions par type, donc les séparer évite l'ambiguïté “est-ce que `copy('dir', 'dir')` voulait dire récursif?”.

Hors v1

- Glob matching en fonction standalone (`luapilot.glob`). Utilise `find` avec `pattern = "*.lua"` pour le cas courant.
- I/O asynchrone. Utilise plutôt [workers](#) pour le traitement parallèle de fichiers.

[English](#) | [Français](#)

luapilot.http — client HTTP

Un client HTTP/HTTPS simple basé sur [cpp-httpplib](#), réutilisant l'OpenSSL embarqué. Synchrone, bloquant, avec timeouts.

Pourquoi

Presque chaque script non trivial doit parler à une API HTTP. Sans client intégré, on en vient à utiliser `curl` via `exec`, avec tout l'overhead de quoting et de spawn de process. `luapilot.http` rend une requête accessible en une ligne, avec la vérification TLS par défaut.

API

Fonction	Renvoie
<code>luapilot.http.request(opts)</code>	<code>response (table) (nil, err)</code>
<code>luapilot.http.get(url, opts?)</code>	raccourci pour <code>request{ method="GET", url=url, ... }</code>
<code>luapilot.http.post(url, opts?)</code>	raccourci pour <code>request{ method="POST", url=url, ... }</code>

Fonction	Renvoie
luapilot.http.put(url, opts?)	raccourci pour request{ method="PUT", url=url, ... }
luapilot.http.delete(url, opts?)	raccourci pour request{ method="DELETE", url=url, ... }

Table opts

Champ	Type	Défaut
url	string (requis pour request)	—
method	string	"GET"
headers	table de name = value	{}
body	string	""
timeout	number (secondes)	30
verify	boolean (vérification cert TLS)	true
ca_cert	string (chemin du fichier CA bundle)	défaut système
ca_path	string (chemin du dossier CA)	défaut système
follow_redirects	boolean	true
max_redirects	integer	5

Table response

```
{
  status = 200,
  body = "...",
  headers = {
    -- clés normalisées en minuscules
    ["content-type"] = "application/json",
    ["content-length"] = "42",
  },
  -- URL finale après redirects :
  url = "https://api.example.com/v1/data",
}
```

Exemples rapides

```
-- GET simple
local r, err = luapilot.http.get("https://api.example.com/health")
if r then
  print(r.status, r.body)
end

-- POST JSON avec header d'auth
local r, err = luapilot.http.post("https://api.example.com/v1/things", {
  headers = {
    ["Content-Type"] = "application/json",
    ["Authorization"] = "Bearer " .. token,
  },
  body = luapilot.json.encode({ name = "widget", count = 7 }),
  timeout = 10,
})

-- Cert auto-signé (dev / CA privée)
local r = luapilot.http.get("https://internal.svc/", {
  ca_cert = "/etc/myapp/internal-ca.crt",
})
```

```
-- Désactiver la vérification (TESTS UNIQUEMENT)
local r = luapilot.http.get("https://expired.badssl.com/",
    { verify = false })
```

Contrat d'erreur

- **Mauvais types d'argument** → lève via `luaL_error`.
- **Erreurs réseau** (DNS, connect, timeout, TLS) → `(nil, "http: <description>")`.
- **Les erreurs de statut HTTP ne sont pas des erreurs** — un **404** renvoie un `response` normal avec `status = 404`. La sémantique du statut est la responsabilité de l'appelant.

TLS / trust store

LuaPilot embarque sa propre OpenSSL statique, donc il n'hérite pas automatiquement de la config `ca-certificates` de la distro. Deux mécanismes garantissent que `verify=true` fonctionne d'emblée :

1. L'OpenSSL embarquée est compilée avec `--openssldir=/etc/ssl`, couvrant Arch, Debian, Ubuntu, Alpine, Gentoo.
2. À l'init TLS, LuaPilot sonde les chemins de CA bundles connus pour Fedora/RHEL, OpenSUSE, FreeBSD, NetBSD.

Tu peux override par appel avec `ca_cert` / `ca_path`, ou globalement via les variables d'environnement `SSL_CERT_FILE` / `SSL_CERT_DIR`. Voir [security](#) et [tls](#) pour les détails.

Décisions de design

- **Synchrone, bloquant**. Les scripts font généralement du request-response one-shot ; le sync est plus simple et suffisant. Pour beaucoup d'appels parallèles, utilise [workers](#).
- **verify=true par défaut**. Désactiver la vérification TLS doit être explicite (`verify=false`). Pas de downgrade silencieux.
- **Le statut HTTP n'est pas une erreur**. L'appelant décide si **404** ou **500** est un échec ; l'appel réseau lui-même a réussi.
- **Headers normalisés en clés minuscules** dans la réponse, pour que les lookups case-insensitive marchent directement (`r.headers["content-type"]`).
- **Suivi des redirects par défaut**. Limité à 5 hops, à override avec `max_redirects` ou à désactiver avec `follow_redirects=false`.

Hors v1

- Streaming du body de réponse (ex : pour télécharger de gros fichiers). Actuellement `body` est lu entièrement en mémoire.
- HTTP/2 / HTTP/3.
- WebSockets (utilise [socket](#) + TLS + une bibliothèque de framing Lua si nécessaire).
- Côté serveur. `cpp-httplib` le supporte, mais exposer un serveur HTTP robuste à Lua est un design à part entière.

[English](#) | **Français**

luapilot.inotify — surveillance d'événements fichier

Wrappe `inotify(7)` de Linux pour la délivrance instantanée d'événements fichier — pas de polling. Utilisé pour le rechargement à chaud de configuration, la surveillance de dossiers de dépôt, le tailing de logs, les triggers de synchronisation de fichiers, etc.

Pourquoi

Poller un dossier avec `listDir` chaque seconde est gaspilleur (CPU + lookups inode) et laggy. `inotify` est le mécanisme dédié du kernel : le kernel ne réveille ton script que quand quelque chose change réellement, avec une latence sub-milliseconde.

L'API Lua expose un contrat minimal sur un syscall notoirement délicat. Le débordement de queue est explicitement remonté (la seule chose pire que de perdre des événements silencieusement, c'est de ne pas te le dire).

API

Fonction	Renvoie
<code>luapilot.inotify.new()</code>	<code>watcher (userdata) (nil, err)</code>
<code>w:add(path, events, opts?)</code>	<code>integer wd (watch descriptor) (nil, err)</code>
<code>w:remove(wd)</code>	<code>(true, nil) (nil, err)</code>
<code>w:read(timeout?)</code>	<code>table d'événements (nil, "timeout") (nil, "interrupted") (nil, err)</code>
<code>w:close()</code>	<code>(true, nil) — idempotent</code>

Argument `events`

Une liste 1..N stricte de strings de noms d'événements. Noms acceptables :

Nom	Déclenché quand
"access"	fichier accédé
"modify"	contenu modifié
"attrib"	métadonnées changées (perms, mtime, ...)
"close_write"	un fichier ouvert en écriture est fermé
"close_nowrite"	un fd read-only est fermé
"open"	fichier ouvert
"moved_from"	fichier déplacé hors du dossier surveillé
"moved_to"	fichier déplacé dans le dossier surveillé
"create"	fichier/dossier créé dans le dossier surveillé
"delete"	fichier/dossier supprimé du dossier surveillé
"delete_self"	le chemin surveillé lui-même est supprimé
"move_self"	le chemin surveillé est déplacé

Événement renvoyé par `read`

```
{  
  wd      = 1,           -- watch descriptor  
  name    = "newfile.txt", -- " si la cible est le chemin surveillé lui-même  
  events  = { create = true, close_write = true },  
  is_dir  = false,  
  cookie  = 0,           -- non-zero apparie moved_from / moved_to  
}
```

Événement synthétique spécial pour le **débordement de queue** :

```
{ wd = -1, events = { overflow = true } }
```

Quand tu vois ça, la queue kernel a manqué de place et des événements ont été perdus. Re-scanner les chemins surveillés pour récupérer l'état.

Exemple rapide

```
local w = assert(luapilot.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

luapilot.signal.handle("TERM", function() w:close(); os.exit(0) end)

while true do
    local events, err = w:read()      -- bloque jusqu'à quelque chose
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify : ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            rescan()                  -- queue saturée, récupère
        else
            handle_new_file("/srv/incoming/" .. ev.name)
        end
    end
end
end
```

Contrat d'erreur

- **read(t)** où t est NaN/Inf/négatif → lève via `luaL_error`.
- **add(path, events)** avec une table `events` non-liste (sparse, clés extra, éléments non-string) → (nil, err).
- **add sur un chemin inexistant** → (nil, "no such file or directory").
- **read :**
 - table `events` en cas de succès.
 - (nil, "timeout") si le timeout est écoulé.
 - (nil, "interrupted") si un signal géré est arrivé.
 - (nil, err) sur autres erreurs.
- **Méthodes après close** → (nil, "inotify: closed").

Décisions de design

- **Non-récurusif en v1.** `inotify(7)` lui-même n'est pas récursif; surveiller un arbre signifie le parcourir et appeler `add` sur chaque sous-dossier, puis gérer les événements `create` pour étendre la surveillance. C'est un vrai travail de design avec des subtilités (races entre le parcours et la délivrance d'événements, gestion du débordement) et méritera un module dédié si/quand nécessaire. Constructible en Lua pur au-dessus.
- **Liste d'événements obligatoire, pas de "all" implicite.** Surveiller tout sature la queue et force chaque événement à traverser Lua. Forcer l'appelant à choisir les événements rend le coût visible.
- **Le débordement est remonté explicitement**, jamais avalé. Le kernel a une queue finie (`/proc/sys/fs/inotify/max_queu` défaut 16384). Quand elle déborde, des événements sont perdus. La seule action appropriée est de re-scanner; perdre silencieusement le signal flinguerait toute la fiabilité.
- **read() est interruptible par signal.** Un `read()` qui bloque pour toujours en ignorant `SIGTERM` est le bug classique de l'arrêt gracieux. Voir [signal](#).
- **IN_CLOEXEC** sur le fd : pas hérité par les sous-process exécutés, cohérent avec les sockets.

Hors v1

- Surveillance récursive. À ajouter au niveau script (`walk + add`) ou attendre une option `recursive = true`.
- Plusieurs watchers par process. Faisable aujourd'hui en créant plusieurs instances `inotify.new()`; pas de registre intégré.
- Sémantique `IN_MASK_ADD` (ajouter des événements à un watch existant). Actuellement `add(path, events)` remplace le mask. Besoin rare.

[English](#) | **Français**

luapilot.json — encode/décode JSON

Basé sur [nlohmann/json](#), la bibliothèque JSON C++ la plus utilisée. Gère le round-trip classique plus la sentinelle “tableau vide” explicite que les tables Lua ne peuvent pas désambiguïser de “objet vide”.

Pourquoi

Le mismatch d’impédance Lua/JSON piège toujours quelqu’un : Lua ne distingue pas une liste vide d’une table vide. La plupart des bibliothèques Lua/JSON ad-hoc choisissent une option et se trompent une fois sur deux. `luapilot.json` est explicite : `{}` se décode en `{}` et se ré-encode en `{}` (objet), mais si tu veux `[]` tu utilises la sentinelle `luapilot.json.empty_array`.

API

Fonction	Renvoie
<code>luapilot.json.encode(value, opts?)</code>	<code>string</code> (texte JSON) <code>(nil, err)</code>
<code>luapilot.json.decode(text)</code>	<code>value</code> <code>(nil, err)</code>
<code>luapilot.json.empty_array</code>	sentinelle qui s’encode en <code>[]</code>

Options d’encodage (table `opts`) :

- `pretty = true` — pretty-print avec indentation (défaut `false`).
- `indent = N` — largeur d’indentation quand `pretty=true` (défaut 2).

Exemple rapide

```
local J = luapilot.json

-- Décode
local data, err = J.decode([[
{ "name": "alice", "tags": ["admin", "ops"], "active": true }
]])
print(data.name, data.tags[1])

-- Encode (compact)
local out = J.encode({ a = 1, b = { 2, 3 } })
-- out == '{"a":1,"b":[2,3]}'

-- Encode pretty
local pretty = J.encode({ a = 1 }, { pretty = true, indent = 4 })

-- Tableau vide vs objet vide
J.encode({})          --> "{}"
J.encode(J.empty_array) --> "[]"
J.encode({ items = J.empty_array }) --> '{"items":[]}'
```

Contrat d’erreur

- **encode** : `(nil, err)` en cas d’erreur d’encodage (cycle dans le graphe, valeur qui ne mappe pas en JSON comme une fonction ou userdata, nombres NaN/Inf, etc.).
- **decode** : `(nil, err)` en cas d’erreur de parsing. Le message d’erreur inclut la ligne/colonne où le parse a échoué.
- **Mauvais type d’argument** → lève via `luaL_error`.

Décisions de design

- **Sentinelle `empty_array`** plutôt que de deviner depuis la structure de la table (certaines bibliothèques traitent `{1, 2, 3}` comme array et `{a=1}` comme objet, ce qui est correct, mais `{}` est ambigu). Une sentinelle explicite enlève toute surprise.
- **NaN/Inf refusés**, pas encodés silencieusement en `nil`. Les consommateurs JSON gèrent `nil` différemment pour les champs numériques ; mieux vaut échouer bruyamment et laisser l'appelant décider.
- **Pas d'API streaming**. Les round-trips sont bornés par la mémoire du script de toute façon ; si tu dois traiter du JSON plus gros que la RAM, utilise un vrai parser streaming dans un process séparé.

Hors v1

- Encodeur/décodeur streaming.
- Un validateur de schéma (utilise-en un en Lua pur — c'est trivial à ajouter au niveau script).

[English](#) | [Français](#)

logging — logger avec niveaux (module Lua)

Un logger standard avec niveaux : `debug`, `info`, `warn`, `error`, `fatal`. Embarqué en tant que module Lua pur, chargé via `require("logging")`. Pas dans le namespace `luapilot` — c'est un module bibliothèque, comme `inspect`.

Pourquoi

Chaque script non trivial finit par réinventer une variante de “niveaux + timestamps + sortie fichier optionnelle”. Standardiser enlève le bikeshed et rend la sortie de log cohérente entre les scripts LuaPilot.

API

```
local log = require("logging")
```

Fonction	Renvoie
<code>log.set_level(name)</code>	<code>(true, nil)</code> — <code>name</code> est l'un de <code>"debug"</code> , <code>"info"</code> , <code>"warn"</code> , <code>"error"</code> , <code>"fatal"</code>
<code>log.set_output(path)</code>	<code>(true, nil)</code> <code>(nil, err)</code> — chemin du fichier (défaut <code>stderr</code>)
<code>log.debug(...)</code> , <code>log.info(...)</code> , <code>log.warn(...)</code> , <code>log.error(...)</code> , <code>log.fatal(...)</code>	chacun affiche si son niveau est <code>>=</code> seuil courant

Les messages sont timestampés (YYYY-MM-DD HH:MM:SS) et taggés avec le niveau (`[DEBUG]`, `[INFO]`, etc.). Plusieurs arguments sont concaténés avec des espaces, comme `print`.

Exemple rapide

```
local log = require("logging")
log.set_level("info")  -- les appels debug en-dessous deviennent no-ops

log.info("démarrage, pid =", luapilot.pid())
log.warn("tentative", 3, "sur", 5)
log.error("connexion échouée :", err)

-- Optionnel : router vers un fichier
local ok, err = log.set_output("/var/log/myapp.log")
if not ok then
    log.fatal("impossible d'ouvrir le log :", err)
```

```
    os.exit(1)
end
```

Sortie :

```
2026-06-11 14:32:01 [INFO] démarrage, pid = 12345
2026-06-11 14:32:05 [WARN] tentative 3 sur 5
2026-06-11 14:32:05 [ERROR] connexion échouée : timeout
```

Contrat d’erreur

- **set_level(name)** : nom inconnu lève via `error()`.
- **set_output(path)** : (nil, err) si le fichier ne peut pas être ouvert en append. Le logging continue vers le sink précédent (stderr si rien n’était défini).
- Les fonctions de logging elles-mêmes n’échouent jamais — si le fichier devient non-écritable en cours de route, le message est silencieusement perdu (l’alternative — crasher le script parce que le volume de log est plein — est pire).

Décisions de design

- **Module Lua pur**. Pas besoin de C++, et être en Lua signifie que les scripts peuvent le monkey-patcher (ex : ajouter une sortie format JSON) au niveau appelant sans recompiler LuaPilot.
- **Cinq niveaux, pas de trace**. Cinq suffisent pour presque tout le monde ; en ajouter d’autres invite chacun à inventer les siens.
- **Sink global unique**. Plusieurs loggers / loggers hiérarchiques / niveaux par module est un piège de feature creep. Si tu as besoin de plusieurs destinations, écris un wrapper.

Hors v1

- Sortie JSON / logging structuré. Facile à bricoler au niveau script si nécessaire.
- Rotation des logs. Utilise `logrotate(8)` sur le fichier de sortie.
- Sortie asynchrone / buffered. Optimisation prématurée pour l’usage typique.

[English](#) | [Français](#)

luapilot.signal — signaux POSIX

Enregistre des callbacks Lua pour les signaux POSIX (`SIGTERM`, `SIGINT`, `SIGHUP`, ...) pour que les scripts long-running puissent s’arrêter proprement ou recharger leur configuration à la demande.

Pourquoi

Les scripts LuaPilot long-running (bots, daemons, watchers) doivent gérer les signaux correctement :

- `SIGTERM` de `systemctl stop` devrait déclencher un arrêt propre.
- `SIGINT` de Ctrl-C devrait faire la même chose.
- `SIGHUP` est le signal conventionnel “reload config”.

Sans handlers explicites, ces signaux tuent juste le process, laissant des fichiers ouverts, des bases de données mi-écrites, et des enfants orphelins.

API

Fonction	Renvoie
<code>luapilot.signal.handle(name, fn)</code>	(true, nil) (nil, err) — installe le callback Lua
<code>luapilot.signal.ignore(name)</code>	(true, nil) (nil, err) — met à <code>SIG_IGN</code>
<code>luapilot.signal.restore_default(name)</code>	(true, nil) (nil, err) — retour au défaut OS
<code>luapilot.signal.kill(pid, name)</code>	(true, nil) (nil, err) — envoie le signal au PID

Fonction	Renvoie
<code>luapilot.signal.list()</code>	table de tous les noms de signaux supportés
<code>luapilot.signal.is_pending()</code>	boolean — y a-t-il un signal géré en file ?

Noms de signaux acceptés (string, casse sensible) :

"HUP", "INT", "QUIT", "USR1", "USR2", "PIPE", "ALRM", "TERM", "CHLD", "CONT", "TSTP", "TTIN", "TTOU", "WINCH". Les autres signaux (KILL, STOP) ne peuvent pas être attrapés — POSIX l'interdit.

Exemple rapide

```

local sig = luapilot.signal
local running = true

sig.handle("TERM", function()
    print("SIGTERM reçu, arrêt en cours")
    running = false
end)
sig.handle("INT", function()
    print("SIGINT reçu, arrêt en cours")
    running = false
end)
sig.handle("HUP", function()
    print("SIGHUP reçu, rechargement config")
    cfg = load_config()
end)

while running do
    local line, err = sock:recv_line(1.0)
    if not line then
        if err == "interrupted" then
            -- Un signal géré est arrivé pendant le recv ; le
            -- callback s'est déjà exécuté. Re-vérifier la condition.
        else
            break
        end
    else
        process(line)
    end
end

print("sortie propre")

```

Comment les callbacks s'exécutent

Quand un signal arrive :

1. Le kernel positionne un flag pending (aucun code Lua ne tourne depuis le signal handler — restrictions async-signal-safe).
2. Si le script est dans un appel bloquant (`recv`, `sleep`, `inotify.read`, `accept`, ...), l'appel renvoie (`nil`, `"interrupted"`).
3. Avant de renvoyer, LuaPilot dispatche tous les callbacks en attente dans l'ordre d'enregistrement, dans le thread Lua principal.
4. Le script continue normalement — le callback a pu modifier des globales (`running = false` est le pattern typique).

Ça veut dire que les callbacks s'exécutent toujours en contexte Lua, jamais depuis l'intérieur d'un signal handler. Ils peuvent utiliser n'importe quelle fonctionnalité Lua (pas de restrictions async-signal-safe).

Contrat d'erreur

- **Nom de signal inconnu** → (nil, "signal: unknown name 'XYZ'").
- **Appels interdits depuis un worker thread** → (nil, "signal: not available in worker threads"). Les signaux sont globaux au process; seul le thread principal les gère.
- **Mauvais types d'argument** → lève via `luaL_error`.
- **kill(pid, name)** pour un PID qui n'existe pas ou ne t'appartient pas → (nil, "kill: ...").

Décisions de design

- **Callbacks tournent dans le thread Lua principal, pas depuis le signal handler.** Les règles async-signal-safety de POSIX interdisent la plupart des opérations utiles depuis un vrai handler. En marquant le signal comme pending et en dispatchant depuis le prochain point sûr, les callbacks peuvent faire tout ce que Lua sait faire.
- **Les appels bloquants renvoient "interrupted" sur un signal géré.** Sans ça, un `recv_line` attendant sur le réseau bloquerait jusqu'au timeout malgré l'arrivée de `SIGTERM`. Avec ça, l'arrêt peut se faire en millisecondes.
- **Pas de ré-entrance.** Pendant qu'un callback tourne, les signaux additionnels sont mis en file et dispatchés ensuite.
- **Interdit dans les worker threads.** Les signaux sont process-wide; seul un thread peut sensément les gérer. Les workers doivent utiliser des channels pour la communication inter-thread.

Hors v1

- `sigprocmask` / masquage fin de signaux. Pas souvent nécessaire au niveau script.
- Intégration avec `signalfd` pour le polling. Le modèle de dispatch actuel est plus simple et couvre les cas typiques.

[English](#) | [Français](#)

luapilot.socket — sockets TCP client et serveur

Sockets TCP bas niveau avec timeouts, conscience des signaux, et un petit set de helpers haut niveau (`recv_line`, `recv_all`). Base pour tout protocole personnalisé — le bot IRC, un daemon JSON-over-TCP, etc. Voir [tls](#) pour la variante chiffrée.

Pourquoi

Pour les besoins HTTP courants il y a [http](#). Mais beaucoup de scripts doivent parler un protocole line-oriented personnalisé (IRC, Redis, daemons de style telnet), où un vrai socket TCP avec timeouts est l'outil approprié. Lua n'a pas de TCP intégré, et `LuaSocket` est une dépendance séparée; `luapilot.socket` rend les patterns courants accessibles en une ligne.

API

Constructeurs

Fonction	Renvoie
<code>luapilot.socket.connect(host, port, opts?)</code>	<code>socket</code> (nil, err)
<code>luapilot.socket.bind(host, port, opts?)</code>	<code>server_socket</code> (nil, err)

`opts` :

- `timeout = N` — timeout par défaut en secondes pour les I/O bloquants (défaut infini).
- `connect_timeout = N` — seulement pour `connect`, défaut 30 s.
- `nodelay = true` — positionne `TCP_NODELAY`.
- `keepalive = true` — positionne `SO_KEEPALIVE`.

Méthodes (socket client)

Méthode	Renvoie
<code>s:send(data)</code>	integer octets envoyés (nil, err)
<code>s:recv(n, timeout?)</code>	string (nil, "timeout") (nil, "interrupted") (nil, err)
<code>s:recv_line(timeout?)</code>	string (sans \n) (nil, ...)
<code>s:recv_all(timeout?)</code>	string (lit jusqu'à EOF) (nil, ...)
<code>s:set_timeout(seconds)</code>	positionne le timeout par défaut (0 = infini)
<code>s:close()</code>	idempotent
<code>s:peer()</code>	(host, port) (nil, err) — endpoint distant

`recv_line` lit jusqu'à trouver \n (LF). CR est strippé s'il est présent (\r\n → renvoyé sans le \r final). A un cap dur de 8 MiB pour empêcher un DoS via une ligne unique sans fin.

Méthodes (socket serveur)

Méthode	Renvoie
<code>srv:accept(timeout?)</code>	socket (nil, ...)
<code>srv:close()</code>	idempotent

Exemples rapides

Client TCP echo

```
local s = assert(luapilot.socket.connect("localhost", 4000, {
    timeout = 5,
    nodelay = true,
}))

s:send("hello\n")
local reply, err = s:recv_line()
print("reçu :", reply)
s:close()
```

Serveur line-protocol simple avec arrêt propre

```
local sig = luapilot.signal
local srv = assert(luapilot.socket.bind("0.0.0.0", 4000))
local running = true
sig.handle("TERM", function() running = false end)
sig.handle("INT", function() running = false end)

while running do
    local client, err = srv:accept(1.0)      -- timeout 1 s
    if not client then
        if err == "timeout" or err == "interrupted" then
            -- boucle et re-vérifie `running`
        else

```

```

        io.stderr:write("accept : ", err, "\n"); break
    end
else
    repeat
        local line = client:recv_line(30)
        if line then client:send("tu as dit : " .. line .. "\n") end
    until not line
    client:close()
end
end
end
srv:close()

```

Contrat d'erreur

- **Erreurs de connect** (DNS, refusé, timeout) → (nil, "socket: ...").
- **Erreurs de bind** (port utilisé, permission) → (nil, "socket: ...").
- **recv** :
 - string (n'importe quelle longueur jusqu'à n demandé).
 - String vide "" quand le peer ferme proprement. Pas une erreur.
 - (nil, "timeout") si pas de données dans le timeout.
 - (nil, "interrupted") si un signal géré est arrivé.
 - (nil, "line too long") pour `recv_line` au-delà du cap 8 MiB.
 - (nil, err) sur autres erreurs.
- **send** : peut renvoyer moins d'octets que demandé. Encapsule dans une boucle si tu dois envoyer une quantité fixe (ou utilise des sémantiques `recv_all`-symétriques dans ton protocole).
- **Méthodes après close** → (nil, "socket: closed").
- **Timeouts NaN/Inf**, timeouts négatifs → lève via `luaL_error`.

Décisions de design

- **Single-threaded, I/O bloquant**. Pas de `select/poll` exposé — utilise `workers` pour les connexions concurrentes, ou des timeouts courts dans une boucle de polling pour les cas simples.
- **recv_line a un cap 8 MiB**. Protège contre un peer qui envoie des mégaoctets sans jamais envoyer `\n`. Configurable plus tard si un cas d'usage demande plus.
- **Blocage conscient des signaux**. Tous les `recv*` et `accept` renvoient "interrupted" sur signal géré, pour que le script puisse sortir proprement.
- **recv renvoie "" sur fermeture propre, pas nil**. Permet aux scripts de distinguer "le distant est parti" ("" de "pas encore de données" ("timeout") sans inventer encore une autre sentinelle.

Hors v1

- Sockets UDP. Facile à ajouter additivement (`luapilot.socket.udp.*`).
- Sockets domaine Unix. Même raisonnement.
- Multiplexage natif (`select/poll/epoll` exposé à Lua). Utilise plutôt les workers ou le polling à timeout court.

[English](#) | [Français](#)

luapilot.sqlite — base de données SQL embarquée

Wrappe SQLite 3.53.1 (embarquée), le moteur de base de données le plus déployé au monde. Persistance zero-config pour tout script qui a besoin de plus qu'un fichier JSON mais moins qu'un serveur de base de données.

Pourquoi

Un script qui doit garder un état entre les runs (la seen-list d'un bot, la progression d'un scraper, des métriques accumulées) ne devrait pas avoir à tirer PostgreSQL ou inventer son propre format de fichier. SQLite est exactement ce qu'il faut à cette échelle : fichier unique, transactionnel, rapide, pas de daemon.

L'API Lua reflète l'API C de SQLite de près (open / exec / prepare / step / finalize) avec des défauts plus sûrs et un retour d'erreurs Lua-friendly.

API

Ouverture et fermeture

Fonction	Renvoie
<code>luapilot.sqlite.open(path, opts?)</code>	<code>db (userdata) (nil, err)</code>
<code>db:close()</code>	<code>(true, nil)</code> — idempotent

`opts` :

Champ	Type	Défaut
<code>readonly</code>	boolean	<code>false</code>
<code>wal</code>	boolean — active le journal WAL	<code>false</code>
<code>busy_timeout</code>	number ms (bloque pendant cette durée sur une db verrouillée)	<code>5000</code>
<code>foreign_keys</code>	boolean	<code>true</code>

Chemin spécial `":memory:"` ouvre une base en mémoire (perdue à la fermeture). À utiliser pour les tests ou le traitement transitoire.

Exec direct (pas de paramètres / instruction one-shot)

Fonction	Renvoie
<code>db:exec(sql)</code>	<code>(true, nil) (nil, err)</code>
<code>db:exec(sql, params)</code>	idem, avec paramètres bindés ?
<code>db:query(sql, params?)</code>	table de tables-de-row <code>(nil, err)</code>

Instructions préparées (réutilisation, performance)

Fonction	Renvoie
<code>db:prepare(sql)</code>	<code>stmt (userdata) (nil, err)</code>
<code>stmt(params?)</code>	iterator sur les rows
<code>stmt:exec(params?)</code>	<code>(true, nil) (nil, err)</code>
<code>stmt:finalize()</code>	<code>(true, nil)</code> — idempotent

Transactions

Fonction	Renvoie
<code>db:transaction(fn)</code>	<code>(true, result)</code> si <code>fn</code> retourne truthy ; <code>rollback + (false, err)</code> sinon

Utilitaires

Fonction	Renvoie
<code>db:last_insert_rowid()</code>	integer

Fonction	Renvoie
<code>db:changes()</code>	<code>integer</code> — rows affectés par la dernière écriture
<code>db:in_transaction()</code>	<code>boolean</code>

Mapping de types

Type SQLite	Type Lua
INTEGER	integer
REAL	number (float)
TEXT	string
BLOB	string (binary-safe)
NULL	nil (lecture), nil ou sentinelle <code>db.NULL</code> (écriture)

Exemples rapides

```

local db = assert(luapilot.sqlite.open("state.db", { wal = true }))

-- Schéma
db:exec([[
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT UNIQUE NOT NULL,
        active INTEGER DEFAULT 1
    );
]])

-- Insert avec paramètres
assert(db:exec("INSERT INTO users (name) VALUES (?)", { "alice" }))
print("inséré avec id", db:last_insert_rowid())

-- Requête de rows
local rows = assert(db:query("SELECT id, name FROM users WHERE active = ?", { 1 }))
for _, r in ipairs(rows) do
    print(r.id, r.name)
end

-- Instruction préparée réutilisée dans une boucle (chemin rapide)
local stmt = assert(db:prepare("INSERT INTO users (name) VALUES (?)"))
for _, name in ipairs({ "bob", "carol", "dave" }) do
    assert(stmt:exec({ name }))
end
stmt:finalize()

-- Transaction
local ok, err = db:transaction(function()
    db:exec("UPDATE users SET active = 0 WHERE name = ?", { "alice" })
    db:exec("UPDATE users SET active = 1 WHERE name = ?", { "bob" })
    return true    -- commit
end)
if not ok then print("rollback :", err) end

db:close()

```

Contrat d'erreur

- Toutes les erreurs runtime → (nil, "sqlite: <description>") avec le code d'erreur SQLite dans le message.
- **exec(sql, params) avec des placeholders ? mais sans argument params** → (nil, "sqlite: placeholders found but no params table provided") — erreur explicite plutôt que binding NULL silencieux, qui est un footgun d'injection classique.
- **Clés sparses dans params** (ex : {[1] = "a", [3] = "c"}) → (nil, err).
- **String BLOB vide** → stockée correctement en tant que BLOB vide (anciennement buggé en pré-1.5).
- **Méthodes après close** → (nil, "sqlite: db closed").
- **Mauvais types d'argument** → lève via `luaL_error`.

Décisions de design

- **WAL opt-in, pas par défaut.** WAL est strictement meilleur pour la plupart des cas d'usage, mais crée des fichiers sidecar `*-wal` et `*-shm` que certains utilisateurs trouvent surprenants. L'opt-in garde le défaut sans surprise, mais tout daemon long-running devrait passer `wal=true`.
- **Clés étrangères ON par défaut**, contrairement au défaut SQLite. La plupart des schémas modernes s'appuient sur des contraintes FK ; les laisser off par défaut est un footgun.
- **transaction(fn) commit si fn retourne truthy.** Convention Lua : retourner une valeur pour signaler le succès, ne rien retourner / nil / false (ou lever) pour rollback. La valeur de retour de la fonction est propagée.
- **Placeholders sans params est une erreur**, pas un bind NULL silencieux. Attrape une classe de bugs proches de l'injection tôt.
- **Erreurs renvoyées, pas levées.** Même un SQL malformé renvoie (nil, err). Les scripts qui ne vérifient pas sont bruyants mais pas catastrophiques.

Hors v1

- I/O streaming BLOB (`sqlite3_blob_open`). Utilise la sérialisation en string si tu rentres en mémoire.
- Virtual tables / FTS5 / R-Tree. Disponibles via SQL brut si le SQLite embarqué est compilé avec ; pas encore d'API niveau Lua.
- API de backup (`sqlite3_backup_init`). Pour l'instant, `VACUUM INTO 'backup.db'` marche en one-liner.

Pour une couche de plus haut niveau type ORM, construis-la en Lua au-dessus de cette API.

[English](#) | **Français**

luapilot strings — manipulation de chaînes

Une seule fonction : découper une chaîne par séparateur en table. Précède la convention par sous-namespace, vit directement sur `luapilot`.

Pourquoi

Découper une chaîne par délimiteur est l'une des opérations string les plus courantes, et la stdlib Lua n'en a pas par défaut (`string.gmatch` marche mais demande d'échapper les patterns). Une fonction dédiée est plus découvrable et évite le piège du pattern escaping.

API

Fonction	Renvoie
<code>luapilot.split(s, sep)</code>	table (array) des sous-chaînes

- `s` : la chaîne à découper.
- `sep` : la chaîne séparateur (littéral, pas de pattern).

Si `sep` n'apparaît pas dans `s`, le résultat est une table à un élément contenant `s` en entier. Si `s` est vide, renvoie une table vide.

Exemple rapide

```
local parts = luapilot.split("a,b,c,d", ",")
-- parts == { "a", "b", "c", "d" }

local one = luapilot.split("hello", ",")
-- one == { "hello" }
```

Contrat d'erreur

- **Mauvais type d'argument** → lève via `luaL_error`.
- Sinon, réussit toujours.

Décisions de design

- **Séparateur littéral, pas un pattern Lua**. Le cas courant est le découpage de données CSV-like, où on veut que `.` signifie un vrai point, pas “n'importe quel caractère”. Si on a besoin de pattern matching, on retombe sur `string.gmatch`.
- **Les entrées vides sont préservées**. `"a,,b"` se découpe en `{"a", "", "b"}`. Les consommateurs qui veulent filtrer les strings vides le font en une ligne de Lua.

Hors v1

- Découpage basé pattern (avec règles d'échappement). Utilise `string.gmatch` si nécessaire.
- Miroir `joinTable(t, sep)` — `table.concat` existe déjà dans la stdlib.

[English](#) | **Français**

luapilot sys — utilitaires système

Helpers d'introspection processus et hôte : PID, hostname, infos kernel, lookup d'exécutable, variables d'environnement.

Pourquoi

La stdlib Lua a `os.getenv` et `os.execute`, mais rien pour le PID, le hostname, les infos kernel, ou un `which` portable. Ce sont des besoins de scripting universels qui ne méritent pas un détour par `popen`.

Ces fonctions vivent directement sur `luapilot` (plat, pas de sous-table) parce qu'elles précèdent la convention par module adoptée pour les ajouts récents.

API

Fonction	Renvoie
<code>luapilot.pid()</code>	<code>integer</code> — PID du processus courant
<code>luapilot.hostname()</code>	<code>string</code> <code>(nil, err)</code> — hostname système
<code>luapilot.uname()</code>	table avec <code>sysname</code> , <code>nodename</code> , <code>release</code> , <code>version</code> , <code>machine</code>
<code>luapilot.which(cmd)</code>	<code>string</code> (chemin absolu) <code>(nil, "not found")</code>
<code>luapilot.env(name)</code>	<code>string</code> <code>nil</code> — comme <code>os.getenv</code> , en cohérent
<code>luapilot.setenv(name, value)</code>	<code>(true, nil)</code> <code>(nil, err)</code>

Exemple rapide

```
print("Sur :", luapilot.hostname(), "PID :", luapilot.pid())

local u = luapilot.uname()
```

```

print("Kernel :", u.sysname, u.release, u.machine)

-- Trouver un binaire
local sh, err = luapilot.which("bash")
if sh then
    luapilot.exec({ sh, "-c", "echo hello" })
end

-- Lire/écrire env
print("HOME :", luapilot.env("HOME"))
luapilot.setenv("MY_VAR", "value")

```

Contrat d’erreur

- **pid()** : ne peut pas échouer, retourne toujours un integer.
- **hostname()** / **which()** : (value, nil) en succès, (nil, err_string) en échec.
- **env(name)** : value si défini, nil sinon. Ne lève jamais.
- **setenv(name, value)** : (true, nil) en succès, (nil, err) en échec (rare — généralement OOM ou nom invalide).
- **uname()** : ne peut pas échouer sur un système supporté, renvoie toujours la table complète.
- **Mauvais type d’argument** (ex : `which(42)`) → lève via `luaL_error` après coercion string par convention Lua. Passe une table ou booléen pour forcer une vraie erreur.

Décisions de design

- **env(name)** renvoie **nil**, pas **""**, pour les variables non définies. Cohérent avec `os.getenv`. Les tests doivent utiliser `env("X") == nil`, pas `env("X") == ""`.
- **which** renvoie le chemin absolu, pas juste “oui/non”. C’est plus utile : on peut `exec` le chemin renvoyé directement. Pour un check booléen, utilise `which(cmd) ~= nil`.
- **uname()** renvoie une table, pas plusieurs valeurs. Reste forward-compatible si un champ est ajouté un jour (ex : `domainname`).

Hors v1

Additif — pourrait être ajouté plus tard :

- `unsetenv` (juste `setenv(name, nil)` pourrait marcher aussi).
- `getuid` / `getgid` / `getppid` / `getlogin`.
- Dump complet de l’environnement en table.

Pour les lookups d’utilisateurs (UID → nom, etc.) voir le module dédié [user](#) — il utilise NSS, qui est le bon backend pour cette question.

[English](#) | [Français](#)

luapilot tables — helpers de manipulation de tables

Deux helpers qui comblerent des manques évidents de la bibliothèque standard Lua : une vraie deep copy, et un merge multi-sources.

Pourquoi

La stdlib Lua n’a pas de deep copy intégrée ni de merge multi-tables. Les deux sont des besoins courants : merger une config defaults + overrides, snapshotter une table avant mutation, copier une arborescence d’options. Les écrire à chaque script est répétitif et source d’erreurs (oublier la metatable, les cycles, etc.).

Ces deux fonctions vivent directement sur la table `luapilot` (pas de sous-namespace) parce qu’elles précèdent la convention par module adoptée pour les ajouts récents.

API

Fonction	Renvoie
<code>luapilot.mergeTables(t1, t2, ...)</code>	nouvelle table — clés string mergées (le dernier écrit gagne), clés numériques append en ordre
<code>luapilot.deepCopyTable(t)</code>	nouvelle table , copiée récursivement

`mergeTables` est variadique — passe autant de tables que tu veux. Les entrées ne sont pas mutées. Le merge a deux règles :

- **Clés string** : les tables suivantes écrasent les précédentes à la même clé (le dernier écrit gagne).
- **Clés numériques (partie array)** : concaténées dans l'ordre où les tables sont passées, puis dans l'ordre 1..n à l'intérieur de chaque table. Donc `mergeTables({1, 2}, {3, 4})` donne `{1, 2, 3, 4}`, pas `{3, 4}`.

`deepCopyTable` copie les tables imbriquées récursivement. Les cycles sont détectés (pas de boucle infinie). Les valeurs non-table (nombres, strings, booleans, fonctions, userdata) ne sont pas dupliquées — elles sont référencées telles quelles.

Exemple rapide

```
-- Clés string : le dernier écrit gagne
local defaults = { port = 8080, host = "localhost", debug = false }
local user_cfg = { port = 9000, debug = true }
local cfg = luapilot.mergeTables(defaults, user_cfg)
-- cfg.port == 9000      (user_cfg a override)
-- cfg.host == "localhost"
-- cfg.debug == true
```

```
-- Clés numériques : append en ordre
local a = { 1, 2 }
local b = { 3, 4 }
local merged = luapilot.mergeTables(a, b)
-- merged == { 1, 2, 3, 4 }  (PAS { 3, 4 })
```

```
-- Deep copy : snapshot avant mutation
local snapshot = luapilot.deepCopyTable(cfg)
cfg.port = 9999      -- N'AFFECTE PAS snapshot
print(snapshot.port)  -- toujours 9000
```

Contrat d'erreur

- **Mauvais type d'argument** (passer une non-table à l'une ou l'autre des fonctions) → lève via `luaL_error`.
- Aucune des deux fonctions ne renvoie d'erreur via `(nil, err)`. Elles réussissent ou lèvent.

Décisions de design

- Les valeurs **shallow** sont référencées, pas **deep-copiées**, dans `deepCopyTable`. Copier des fonctions ou des userdata n'a pas de sens, et copier des valeurs immuables (nombres, strings, booleans) ne change rien. Seules les tables sont dupliquées récursivement.
- Les **metatables** ne sont pas copiées dans `deepCopyTable`. C'est un choix conscient : préserver une metatable à travers une copie peut surprendre l'appelant (la copie déclenche encore des callbacks `__index` qui mutent l'original). Si tu dois copier avec metatable, fais-le explicitement avec `setmetatable(luapilot.deepCopyTable(t), getmetatable(t))`.
- **mergeTables** est **shallow**. Si deux tables sources partagent une sous-table à la même clé string, le résultat référence la dernière telle quelle — il ne récurse pas dedans. Combine avec `deepCopyTable` si tu veux une sémantique merge-puis-isole.

- **Les clés numériques sont append, pas écrasées.** Ce choix vient du cas d'usage Lua le plus courant : combiner des tables type liste ($\{1, 2\} + \{3, 4\} \rightarrow \{1, 2, 3, 4\}$). Si tu voulais une sémantique d'écrasement sur les clés numériques, construis une table de destination et assigne explicitement.

Hors v1

Additif — pourrait être ajouté plus tard sans casser le SemVer :

- Un `deepMergeTables` qui récurse dans les tables imbriquées quand les deux sources ont une table à la même clé. Demande courante, mais la sémantique autour des listes vs maps devient ambiguë, d'où pas dans v1.
- Un `equalsTables` pour comparaison structurelle profonde. Facile à écrire en Lua pur, n'a pas émergé comme besoin réel.

[English](#) | **Français**

luapilot.time — horloges monotone et temps réel

Deux horloges avec précision sub-seconde et un `sleep` qui respecte les unités `s/ms/us/ns` ainsi que les signaux. Comble les trous de `os.time` / `os.clock`.

Pourquoi

`os.time()` ne donne que des secondes, et `os.clock()` mesure le temps CPU, pas le temps réel. Aucun des deux n'est approprié pour mesurer des durées réelles (latence de requête, backoff de retry, timeouts). Et `os` n'a pas de `sleep` portable à la nanoseconde.

`luapilot.time` expose `CLOCK_MONOTONIC` (durées) et `CLOCK_REALTIME` (timestamps), avec `sleep()` qui respecte l'interruption par signal.

API

Fonction	Renvoie
<code>luapilot.time.monotonic()</code>	<code>number</code> — secondes depuis une epoch arbitraire (immune aux changements d'horloge)
<code>luapilot.time.realtime()</code>	<code>number</code> — temps POSIX (secondes depuis 1970-01-01 UTC)
<code>luapilot.sleep(amount, unit?)</code>	<code>(true, nil) (nil, "interrupted")</code>

`unit` pour `sleep` est l'un de `"s"` (défaut), `"ms"`, `"us"`, `"ns"`. Les floats sont acceptés.

Exemple rapide

```
-- Mesurer une durée en toute sécurité (immune aux sauts NTP / DST)
local t0 = luapilot.time.monotonic()
do_something()
local elapsed = luapilot.time.monotonic() - t0
print(string.format("durée %.3f s", elapsed))

-- Timestamper un événement
local now = luapilot.time.realtime()
db:exec("INSERT INTO events (ts, kind) VALUES (?, ?)", { now, "ping" })

-- Dormir 100 ms, mais se réveiller sur un signal
local ok, err = luapilot.sleep(100, "ms")
if not ok and err == "interrupted" then
    print("réveillé par un signal")
end
```

Contrat d'erreur

- **monotonic()** / **realtime()** : ne peuvent pas échouer. Renvoient toujours un number.
- **sleep(amount, unit)** :
 - (true, nil) si le sleep s'est terminé normalement.
 - (nil, "interrupted") si un signal géré est arrivé pendant le sleep (voir [signal](#)).
- NaN / Inf / **amount négatif** → lève via luaL_error.
- **String d'unit inconnue** → lève via luaL_error.

Décisions de design

- **Deux horloges, deux fonctions, pas de flag**. **monotonic()** pour les durées, **realtime()** pour les timestamps. Les fondre en une fonction avec paramètre inviterait au mauvais choix.
- **sleep conscient des signaux**. Un sleep de 60 secondes qui ignore SIGTERM est le bug classique de l'arrêt gracieux. **sleep()** renvoie (nil, "interrupted") pour que l'appelant puisse sortir proprement.
- **Pas de helper "format date"**. **os.date(format, realtime())** marche très bien, et une fonction dupliquée ne serait qu'un wrapper.

Hors v1

- **luapilot.time.boottime()** (CLOCK_BOOTTIME — inclut le suspend). Rarement utile, facile à ajouter plus tard.
- **Helpers luapilot.time.utcnow()** / **localnow()** qui renvoient des strings pré-formatées. Trivial en Lua, sans valeur ajoutée.

[English](#) | **Français**

luapilot.socket TLS — connect_tls et starttls

La moitié TLS de [socket](#) : TCP chiffré pour IRC over TLS, IMAPS, protocoles sécurisés personnalisés. Deux points d'entrée : **connect_tls** pour les protocoles qui font TLS dès le départ (port 6697 IRC, 993 IMAPS, etc.) et **starttls** pour les protocoles qui upgradent une connexion en clair existante (SMTP, IMAP, IRC STARTTLS).

Pourquoi

Parler du TCP chiffré sans [http](#) est un vrai besoin : bots IRC sur +6697, soumission SMTP, services internes custom derrière TLS. Le faire manuellement avec openssl s_client via exec n'est pas fiable ; la bibliothèque C++ openssl + les abstractions choisies de cpp-httplib donnent une API Lua propre qui gère la vérification de certs correctement par défaut.

API

Fonction	Renvoie
luapilot.socket.connect_tls(host, port, opts?)	tls_socket (nil, err)
s:starttls(opts?)	(true, nil) (nil, err) — upgrade un socket en clair existant

opts (fusionnés avec les opts socket habituels) :

Champ	Type	Défaut
verify	boolean	true
ca_cert	string (chemin)	bundle système
ca_path	string (chemin)	bundle système
server_name	string	argument host
alpn	array de strings	aucun

Un `tls_socket` a les mêmes méthodes qu'un socket en clair (`send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `close`, `peer`). Le chiffrement est transparent.

Exemples rapides

Connexion à IRC over TLS

```
local s = assert(luapilot.socket.connect_tls("irc.libera.chat", 6697, {
    timeout = 30,
}))
```

```
s:send("NICK luapilot-bot\r\n")
s:send("USER luapilot 0 * :luapilot bot\r\n")
```

```
while true do
    local line, err = s:recv_line()
    if not line then break end
    print(line)
    if line:match("^PING (.+)$") then
        s:send("PONG " .. line:match("^PING (.+)$") .. "\r\n")
    end
end
```

Upgrade STARTTLS (soumission SMTP)

```
local s = assert(luapilot.socket.connect("mail.example.com", 587))
print(s:recv_line())           -- bannière 220
s:send("EHLO myhost\r\n")
repeat
    line = s:recv_line()
until not line:match("^250%-") -- dernière ligne 250

s:send("STARTTLS\r\n")
print(s:recv_line())           -- 220 ready to start TLS

assert(s:starttls({ server_name = "mail.example.com" }))
-- s est maintenant chiffré ; continue avec EHLO + AUTH + ...
```

Serveur auto-signé (dev / CA privée)

```
local s = assert(luapilot.socket.connect_tls("internal.svc", 5555, {
    ca_cert = "/etc/myapp/internal-ca.crt",
}))
```

Skip de la vérification (TESTS UNIQUEMENT)

```
local s = assert(luapilot.socket.connect_tls("self-signed.local", 8443, {
    verify = false,
}))
```

Contrat d'erreur

- Toutes les erreurs préfixées `"tls: ..."` (handshake, cert verify, etc.) ou `"socket: ..."` (DNS, connect, timeout).
- `(nil, err)` toujours — pas d'exceptions pour les échecs "attendus" comme un cert mismatch.
- **verify=true + cert invalide** → `(nil, "tls: certificate verify failed: <reason>")`. Raisons courantes : expiré, auto-signé, hostname mismatch, CA inconnue.
- **Timeouts NaN/Inf**, mauvais types → lève via `luaL_error`.

Trust store

LuaPilot embarque sa propre OpenSSL statiquement liée. D'emblée, `verify=true` fonctionne sur :

- **Arch, Debian, Ubuntu, Alpine, Gentoo** : via `--openssldir=/etc/ssl` baké dans l'OpenSSL embarquée.
- **Fedora, RHEL, CentOS, Rocky, Alma, OpenSUSE, FreeBSD, NetBSD** via sondage runtime des chemins de CA bundles connus.

Overrides, par ordre de priorité :

1. `ca_cert` / `ca_path` par appel.
2. Variables d'env `SSL_CERT_FILE` / `SSL_CERT_DIR`.
3. Les deux mécanismes ci-dessus.

Si aucun ne trouve de CA store et que tu utilises `verify=true`, le handshake échoue avec une erreur claire. Voir [security](#) pour le tableau complet.

Décisions de design

- **TLS 1.2 minimum**. SSL 3, TLS 1.0, TLS 1.1 sont inconditionnellement refusés — ils sont cassés, aucun serveur moderne ne devrait s'y reposer. TLS 1.3 est négocié automatiquement quand disponible.
- **`verify=true` par défaut, `verify=false` exige un opt-in explicite**. Aucun moyen de désactiver accidentellement la vérification de cert.
- **`server_name` défaut sur l'argument `host`** pour que SNI marche tout seul. Passe-le explicitement si tu te connectes par IP à un serveur TLS qui utilise SNI.
- **Mêmes méthodes que socket en clair**, donc une fonction qui prend un `socket` fonctionne avec les deux de manière transparente.

Hors v1

- Authentification client par certificat. Possible à ajouter plus tard en option `client_cert` / `client_key`.
- TLS session resumption / session tickets. Pas courant au niveau script.
- Vérification OCSP stapling. Reposition sur les défauts d'OpenSSL.

[English](#) | **Français**

luapilot.toml — parseur TOML

Basé sur [toml++](#), un parser TOML 1.0 strict. Décodage seulement — encoder du TOML fidèlement depuis une table Lua est ambigu (Lua ne distingue pas les inline tables et les dotted tables, par exemple).

Pourquoi

TOML est le format de config naturel des outils modernes (Cargo, pyproject, systemd-credentials, etc.). Un script LuaPilot lisant sa propre config devrait pouvoir le faire directement, pas via une conversion TOML → JSON via `tomlq`.

API

Fonction	Renvoie
<code>luapilot.toml.decode(text)</code>	<code>table</code> (TOML parsé) <code>(nil, err)</code>

La table renvoyée reflète la structure TOML :

- Strings → strings Lua (UTF-8)
- Integers → integers Lua
- Floats → numbers Lua

- Booleans → booleans Lua
- Datetimes → strings Lua au format ISO 8601 (TOML n'a pas d'équivalent natif Lua ; utilise `os.date` ou une bibliothèque date si tu as besoin des composants parsés)
- Arrays → tables Lua (1-indexées)
- Tables (`[section]` ou `{ inline }`) → tables Lua (avec clés strings)

Exemple rapide

```
local body = [[
title = "My App"
[server]
host = "localhost"
port = 8080
features = ["auth", "metrics"]

[database]
url = "sqlite:///var/lib/app.db"
]]

local cfg, err = luapilot.toml.decode(body)
if not cfg then error("parsing config échoué : " .. err) end

print(cfg.title)           -- "My App"
print(cfg.server.host)     -- "localhost"
print(cfg.server.features[1]) -- "auth"
print(cfg.database.url)    -- "sqlite:///var/lib/app.db"
```

Contrat d'erreur

- **decode** : (nil, err) en cas d'erreur de parse. Le message d'erreur inclut la ligne/colonne où le parse a échoué, et une courte description de ce qui n'allait pas (par `toml++`).
- **Argument non-string** → lève via `luaL_error`.

Décisions de design

- **Décode seulement, pas d'encode**. Round-tripper un fichier TOML via une table Lua perd la mise en forme (commentaires, dotted vs inline tables, newlines entre sections) que les vrais utilisateurs apprécient. Si tu dois écrire du TOML, construis la string toi-même — TOML est conçu pour être écrit à la main. Un encodeur fidèle serait un gros chantier pour une valeur limitée.
- **Datetimes en strings**. Le type datetime TOML est riche (offset, local, date-only, time-only) mais Lua n'a pas d'équivalent natif. Renvoyer une string ISO 8601 préserve la donnée sans perte et laisse le script la parser avec les outils de son choix.

Hors v1

- Encodeur. Pourrait être ajouté si un vrai cas d'usage apparaît, mais peu probable.
- Un validateur de schéma. Construis-en un en Lua si nécessaire ; la structure décodée n'est qu'une table.

[English](#) | **Français**

luapilot.user

Lookups d'utilisateurs système via NSS, sans parser `/etc/passwd` directement.

Pourquoi

Sur un système Linux moderne, les utilisateurs peuvent venir de plusieurs sources : `/etc/passwd`, LDAP, SSSD, NIS+, FreeIPA, systemd-userdb, ou d'autres plugins NSS. Lire `/etc/passwd` directement rate tout sauf la première source. `luapilot.user` s'appuie sur `getpwnam_r(3)` et `getpwuid_r(3)`, qui traversent la chaîne de résolveurs configurée dans `/etc/nsswitch.conf` — exactement comme `id` ou `getent passwd` le font.

API

Fonction	Renvoie
<code>user.get(name_or_uid)</code>	<code>table (nil, "user not found") (nil, "user: <err>")</code>
<code>user.exists(name_or_uid)</code>	<code>boolean</code>

`name_or_uid` accepte :

- une **string** → lookup par nom via `getpwnam_r`
- un **integer non-négatif** → lookup par UID via `getpwuid_r`

Tout autre type, un float, un integer négatif, une string contenant un NUL embarqué, ou un integer supérieur à `uid_t` max lèvent via `luaL_error` — ce sont des bugs côté appelant, pas des conditions runtime à gérer proprement.

Table renvoyée en cas de succès

```
{
  name  = "yaourt",
  uid   = 968,
  gid   = 968,
  gecos = "yaourt AUR build user",
  home  = "/var/cache/yaourt",
  shell = "/usr/sbin/nologin",
}
```

Tous les champs string sont garantis non-nil. Ils peuvent être des strings vides quand la source NSS sous-jacente n'a pas de valeur (c'est rare mais possible — typiquement un `gecos` vide).

Exemple rapide

```
-- S'assurer qu'un user système est provisionné avant de lancer
-- le daemon.
if not luapilot.user.exists("yaourt") then
  error("user yaourt absent – fais d'abord useradd")
end

local u = assert(luapilot.user.get("yaourt"))
luapilot.chdir(u.home)
```

Contrat d'erreur

- **Mauvais type d'argument** (table, boolean, nil, float, integer négatif, UID au-dessus de `uid_t` max, string avec NUL embarqué) → lève via `luaL_error`. Utilise `pcall` si tu dois récupérer.
- **Utilisateur absent** (le résolveur ne renvoie aucun match) → `get()` renvoie `(nil, "user not found")`; `exists()` renvoie `false`.
- **Erreur NSS** (le résolveur lui-même est cassé : LDAP injoignable, mémoire saturée, etc.) → `get()` renvoie `(nil, "user: <description>")`; `exists()` renvoie `false` (les erreurs sont silencieusement assimilées à “absent” pour les prédicats; utilise `get()` si tu dois diagnostiquer).

Décisions de design

- **NSS uniquement, jamais /etc/passwd** : lire le fichier directement raterait silencieusement tous les comptes qui ne sont pas dans le fichier local. Tout l'intérêt de `luapilot.user` est que le script obtienne la même réponse que `id` ou `getent passwd`.
- **Variantes `_r` thread-safe** : `LuaPilot` expose `luapilot.workers`, donc on utilise `getpwnam_r/getpwuid_r` partout.
- **gecos exposé brut** : le format historique est `Full Name,Office,WorkPhone,HomePhone[,Other]`, mais en pratique la plupart des entrées contiennent juste un nom libre. Parser ça en Lua est trivial si nécessaire ; baker un parser en C++ serait prématuré.
- **Rejet du NUL embarqué** : `"root\0evil"` serait vu comme `"root"` par `getpwnam_r` (qui s'arrête au premier NUL). Sur de l'input dérivé d'un utilisateur, ça pourrait contourner une vérification d'identité en amont. Le rejet explicite est plus sûr que la troncature implicite.
- **Check du range `uid_t`** : sur Linux, `uid_t` est `uint32_t`, alors que `lua_Integer` est `int64_t`. Sans check, `5_000_000_000` serait silencieusement tronqué à 32 bits et pourrait matcher un compte non lié.

Hors v1

Additif — ces choses pourraient être ajoutées plus tard sans casser le SemVer :

- Lookups de groupes (`getgrnam` / `getgrgid`) — sera ajouté quand un cas concret le demandera.
- Accès à `/etc/shadow` (mots de passe, expiration) — hors scope. Exige root et est rarement utile hors scripts admin de niche. L'authentification mot de passe devrait passer par PAM, pas par `LuaPilot`.
- Création d'utilisateurs/groupes — déjà faisable via `luapilot.exec("useradd ...")` et `groupadd`. Pas besoin d'un binding dédié.

[English](#) | [Français](#)

luapilot.workers — jobs parallèles

Exécute du code Lua sur de vrais threads OS. Utile pour le travail I/O-bound qui bénéficie de la concurrence (fetcher beaucoup d'URLs, scanner beaucoup de fichiers, traiter des batches) et le travail CPU-bound sur des machines multi-cœurs.

Pourquoi

Lua a des coroutines, mais elles tournent coopérativement sur un seul thread OS — très bien pour les machines à états, pas pour utiliser plusieurs cœurs. Et beaucoup de tâches sont I/O-bound : 100 requêtes HTTP qui prennent 500 ms chacune mettent quand même 50 s séquentiellement, mais ~500 ms en parallèle.

`luapilot.workers` spawn de vrais threads OS, chacun faisant tourner son propre état Lua isolé, communiquant avec le parent via les arguments passés et les valeurs de retour. Pas d'état partagé signifie pas de locks, pas de data races — juste envoyer du travail, récupérer des résultats.

API

Fonction	Renvoie
<code>luapilot.workers.spawn(code, args?)</code>	<code>job (userdata) (nil, err)</code>
<code>luapilot.workers.cpu_count()</code>	<code>integer</code> — nombre de CPUs logiques
<code>job:join(timeout?)</code>	<code>(true, result) (false, err) (nil, "timeout")</code>
<code>job:done()</code>	<code>boolean</code> — check non-bloquant
<code>job:cancel()</code>	<code>(true, nil)</code> — coopératif; positionne un flag

Argument code

Une string source Lua qui tourne dans le worker. Le namespace complet `luapilot` est disponible, plus une globale `worker` avec :

- `worker.args` — le deuxième argument passé à `spawn`.
- `worker.cancelled()` — renvoie `true` si le parent a appelé `job:cancel()`. Le worker est censé vérifier ça dans les boucles longues.

La valeur de la dernière expression du worker (ou `return value`) est ce que `join()` renvoie comme `result`.

Argument `args`

Toute valeur qui peut être deep-copiée entre états Lua : `nil`, booléens, numbers, strings, tables des mêmes. **Pas** : fonctions, userdata, threads, tables avec cycles. Les tables sont deep-copiées — pas de partage.

Exemples rapides

Fetches HTTP parallèles

```
local urls = {
    "https://a.example/",
    "https://b.example/",
    "https://c.example/",
    "https://d.example/",
}

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = luapilot.workers.spawn([[
        local url = worker.args.url
        local r, err = luapilot.http.get(url, { timeout = 10 })
        if not r then return { ok = false, err = err } end
        return { ok = true, status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    if ok then
        print(i, result.status or result.err)
    else
        print(i, "worker crashed :", result)
    end
end
```

Travail CPU-bound avec cancellation

```
local n = luapilot.workers.cpu_count()
print("on tourne sur", n, "cœurs")

local jobs = {}
for i = 1, n do
    jobs[i] = luapilot.workers.spawn([[
        local chunk = worker.args.chunk
        local total = 0
        for x = chunk.from, chunk.to do
            if x % 1000 == 0 and worker.cancelled() then
                return { cancelled = true, partial = total }
            end
            total = total + heavy_compute(x)
        end
        return { total = total }
    ]])
end
```

```

    ]], { chunk = { from = i * 1000, to = (i + 1) * 1000 - 1 } })
end

-- ... plus tard, si nécessaire :
-- for _, j in ipairs(jobs) do j:cancel() end

```

Pattern pool de workers

```

local function map_parallel(items, code, max_concurrent)
    max_concurrent = max_concurrent or luapilot.workers.cpu_count()
    local results = {}
    local i = 1
    local active = {}

    while i <= #items or next(active) do
        -- en lancer autant que possible
        while i <= #items and #active < max_concurrent do
            active[#active + 1] = {
                index = i,
                job = luapilot.workers.spawn(code, { item = items[i] }),
            }
            i = i + 1
        end
        -- moissonner les finis
        for k = #active, 1, -1 do
            local entry = active[k]
            if entry.job:done() then
                local ok, result = entry.job:join()
                results[entry.index] = ok and result or { err = result }
                table.remove(active, k)
            end
        end
        if next(active) then luapilot.sleep(10, "ms") end
    end
    return results
end

```

Contrat d'erreur

- **spawn** : (nil, err) si le thread OS ne peut pas être créé (rare — généralement des limites de ressources).
- **join** :
 - (true, result) — worker terminé normalement; result est sa valeur de retour (ou nil s'il n'a pas retourné).
 - (false, err) — worker crashé avec une erreur non attrapée; err est le message d'erreur + traceback.
 - (nil, "timeout") — timeout écoulé sans que le worker finisse.
- **cancel** n'échoue jamais. Le flag est positionné; le worker peut prendre un moment à le remarquer (il doit appeler worker.cancelled()).
- **Mauvais types d'argument** → lève via luaL_error.

Partage de données

Les workers ne partagent pas l'état Lua. La communication se fait :

- **En entrée** : via l'argument args de spawn (deep-copié dans l'état du worker).
- **En sortie** : via la valeur de retour du worker (deep-copiée en retour).
- **Effets de bord** : un worker peut écrire dans un fichier, appender dans un log, requêter une base de données
- mais la coordination via ces effets de bord est la responsabilité du script.

Si deux workers écrivent dans le même fichier SQLite, positionne `wal=true` à `open` pour qu'ils ne se verrouillent pas mutuellement. SQLite gère la synchronisation correctement avec WAL; dans `busy_timeout` tu peux configurer combien de temps attendre sur une db occupée.

Décisions de design

- **Un état Lua par worker, pas de partage.** L'alternative — état partagé avec locks — est une source bien connue de bugs subtils et on ne pense pas que les locks niveau Lua valent le travail de design à ce niveau. Le message-passing pur est plus simple.
- **Pas de thread pool global.** Chaque `spawn` crée un nouveau thread OS, chaque `join` le ferme. Pour les boucles serrées, construis un pool en Lua (voir l'exemple ci-dessus). Le modèle plus simple gagne en lisibilité.
- **Cancellation coopérative, pas préemptive.** Il n'y a pas moyen d'arrêter de force un thread Lua au milieu d'une opération sans laisser le runtime dans un état indéfini. `worker.cancelled()` renvoie `true`; le worker est responsable de le vérifier périodiquement.
- **luapilot.signal n'est pas disponible dans les workers.** Les signaux sont process-wide; seul le thread principal peut les gérer sensément.

Hors v1

- Channels (queues FIFO entre workers). À ajouter plus tard si un cas d'usage montre que le pattern est courant.
- Mémoire partagée (mmap). Idem.
- Futures I/O async (style `spawn().then(...)`). Hors scope; utilise une boucle de polling ou des checks `done()`.

[English](#) | **Français**

Cookbook

Recettes concrètes pour des tâches de scripting courantes. Chaque recette est courte et autonome; copie, adapte, déploie.

Surveiller un dossier et envoyer les nouveaux fichiers par email

Un dossier de dépôt qui mailise chaque fichier dès qu'il arrive. Utilise `luapilot.inotify` pour le réveil instantané (pas de polling), et écoute `close_write` + `moved_to` pour garantir que le fichier est complet.

```
local w = assert(luapilot.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

luapilot.signal.handle("TERM", function()
    w:close()
    os.exit(0)
end)

while true do
    local events, err = w:read()
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            -- queue saturée : rescanne le dossier
            -- (des événements ont été perdus)
        else
            local path = "/srv/incoming/" .. ev.name
            send_email(path)
        end
    end
end
```

```

        os.remove(path)
    end
end
end

```

S'assurer qu'un user système existe, puis basculer dessus

Check typique à l'install avant de lancer un daemon sous un utilisateur non-root.

```

local function ensure_user(name)
    if luapilot.user.exists(name) then return true end
    local ok, err = luapilot.exec({
        "useradd",
        "--system",
        "--home-dir", "/var/lib/" .. name,
        "--create-home",
        "--shell", "/usr/sbin/nologin",
        name,
    })
    if not ok then
        return nil, "useradd a échoué : " .. tostring(err)
    end
    return true
end

assert(ensure_user("myapp"))
local u = assert(luapilot.user.get("myapp"))
luapilot.chdir(u.home)

```

Récupérer beaucoup d'URLs en parallèle

luapilot.workers fait tourner du Lua sur plusieurs cœurs. Le temps total est proche du fetch le plus lent, pas de la somme.

```

local urls = { "https://a.example/", "https://b.example/",
               "https://c.example/", "https://d.example/" }

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = luapilot.workers.spawn([[
        local url = worker.args.url
        local r, err = luapilot.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end

```

Arrêt propre d'un script long-running

Attraper SIGTERM / SIGINT pour que le script puisse fermer ses ressources proprement. Fonctionne avec systemctl stop et Ctrl-C.

```

local log = require("logging")
log.set_level("info")

```



```

local running = true
luapilot.signal.handle("TERM", function()
    log.info("SIGTERM, arrêt en cours")
    running = false
end)
luapilot.signal.handle("INT", function()
    log.info("Ctrl-C, arrêt en cours")
    running = false
end)

while running do
    -- boucle principale ; les appels bloquants renvoient
    -- (nil, "interrupted") quand un signal géré arrive, donc la
    -- boucle sort rapidement.
    local line, err = sock:recv_line()
    if not line then
        if err == "interrupted" then break end
        log.error("recv : ", err); break
    end
    process(line)
end

sock:close()
log.info("arrêt propre")

```

Lire une config TOML, valider, persister en SQLite

```

-- Charger le TOML
local f = assert(io.open("config.toml", "r"))
local body = f:read("a"); f:close()
local cfg, perr = luapilot.toml.decode(body)
if not cfg then error("config.toml : " .. perr) end

-- Valider (checks manuels simples ; pour un vrai schéma, bâtis-le en Lua)
assert(type(cfg.server) == "table", "section [server] absente")
assert(type(cfg.server.host) == "string", "server.host absent")
assert(math.type(cfg.server.port) == "integer", "server.port doit être un entier")

-- Ouvrir la db, persister
local db = assert(luapilot.sqlite.open("state.db", { wal = true }))
db:exec([[CREATE TABLE IF NOT EXISTS config (k TEXT PRIMARY KEY, v TEXT)])]
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
    { "host", cfg.server.host })
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
    { "port", tostring(cfg.server.port) })
db:close()

```

Pretty-print d'une valeur Lua

Le module `inspect` (embarqué) donne une sortie lisible pour les tables imbriquées.

```

local inspect = require("inspect")
print(inspect({ a = 1, b = { c = 2, d = { 3, 4, 5 } } }))
-- { a = 1, b = { c = 2, d = { 3, 4, 5 } } }

```

Hello there

Le script LuaPilot minimum :

```
luapilot.helloThere()
```